

Desarrollo de Contratos Inteligentes y Aplicaciones Descentralizadas



Israel Jaudy Perez Bermudez
Armando Rivera Castillo



**Quinta Reunion de Ciberseguridad para la Industria 5.0
- RCI 5.0**

Instituto Nacional de Astrofísica, Óptica y Electrónica

Sesiones: 7 y 8 de octubre de 2026

15:00–18:30 h

Objetivo del taller

Al finalizar las dos sesiones, los participantes habrán recorrido el proceso completo de desarrollo de un contrato inteligente:

- ▶ Comprender el papel de blockchain en una DApp.
- ▶ Comprender como se ejecutan transacciones y contratos inteligentes en Ethereum.
- ▶ Programar un contrato inteligente en Solidity.
- ▶ Compilarlo, desplegarlo y probarlo en Remix IDE.
- ▶ Observar eventos, validar el control de acceso y analizar el consumo de gas.

Caso practico

Registro y verificación
descentralizada
de documentos

Desarrollar → Desplegar →
Probar

Estructura del taller

Dia 1 - Fundamentos

- ▶ Blockchain y confianza digital.
- ▶ Hashes, firmas y transacciones.
- ▶ Ethereum, EVM, cuentas y gas.
- ▶ DApps y contratos inteligentes.
- ▶ Fundamentos de Solidity.

Dia 2 - Taller practico

- ▶ Abrir el material en GitHub y Remix.
- ▶ Compilar y desplegar en Remix VM.
- ▶ Registrar, consultar y revocar documentos.
- ▶ Probar control de acceso.
- ▶ Revisar eventos, transacciones y gas.

Idea central

El primer dia construye el marco conceptual. El segundo dia convierte ese marco en una practica ejecutable de inicio a fin.

Conectividad para el taller

Red Wi-Fi

Nombre de la red

RCI

Contraseña

inaoe2020

Verificación rápida

Antes de continuar, comprobar que pueden acceder a:

- ▶ Internet.
- ▶ GitHub.
- ▶ Remix IDE.

No es necesario instalar software.

Durante la práctica

El acceso a Internet será necesario para consultar el repositorio del taller y trabajar con Remix IDE desde el navegador.

Conceptos clave antes de iniciar

Blockchain

Registro compartido y verificable donde las operaciones se agrupan en bloques enlazados criptográficamente.

Solidity

Lenguaje de programación usado para escribir contratos inteligentes compatibles con Ethereum.

Ethereum

Plataforma blockchain que permite ejecutar contratos inteligentes y aplicaciones descentralizadas.

Remix IDE

Entorno web para escribir, compilar, desplegar y probar contratos sin instalar herramientas locales.

EVM

Ethereum Virtual Machine: entorno que ejecuta el código compilado de los contratos.

Gas

Medida del costo computacional de ejecutar operaciones en Ethereum.

Conceptos de uso y aplicacion

Contrato inteligente

Programa desplegado en blockchain que define reglas, valida condiciones, modifica estado y emite eventos.

DApp

Aplicacion descentralizada que combina una interfaz tradicional con cuentas, contratos inteligentes y blockchain.

Transaccion

Operacion firmada por una cuenta que puede modificar el estado de la blockchain.

Cuenta / Wallet

Identidad criptografica desde la cual se firman operaciones. En Remix usaremos cuentas de prueba.

Evento

Registro emitido por un contrato para dejar evidencia observable de una accion importante.

Funcion view

Funcion de lectura que consulta estado sin modificarlo; normalmente no requiere una transaccion externa.

Identidad digital autosoberana como ejemplo

SSI

La *Self-Sovereign Identity* busca que las personas controlen sus identificadores y credenciales digitales.

DID

Un identificador descentralizado permite referenciar una identidad sin depender exclusivamente de un registro central.

Credencial verificable

Afirmación digital que puede presentarse y verificarse criptográficamente.

Relacion con blockchain

Blockchain no tiene que almacenar toda la credencial; puede registrar información mínima:

- ▶ llaves públicas;
- ▶ hashes;
- ▶ estados de revocación;
- ▶ referencias a datos off-chain.

Relacion con el taller

Usaremos la misma idea: registrar información verificable sin guardar documentos completos en blockchain.

Principios basicos que guiaran el taller

1. Verificabilidad

Los participantes deben poder comprobar datos y operaciones sin depender totalmente de una sola entidad.

2. Integridad

Un cambio en la informacion debe ser detectable mediante mecanismos criptograficos.

3. Trazabilidad

Las operaciones relevantes deben quedar observables mediante transacciones y eventos.

4. Minimizacion de datos

No todo debe guardarse on-chain. En muchos casos basta almacenar hashes, estados o referencias.

5. Control de acceso

El contrato debe definir claramente quien puede ejecutar cada accion.

6. Reproducibilidad

La practica debe poder ejecutarse desde el navegador con GitHub, Remix IDE y Remix VM.

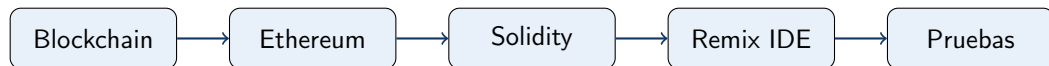
Distribucion del tiempo

Sesion	Tiempo	Enfoque
Dia 1: fundamentos de blockchain, Ethereum y Solidity	3.5 h	Comprender el contexto, los conceptos y el diseno del caso practico.
Dia 2: practica guiada en Remix IDE	3.5 h	Implementar, compilar, desplegar, probar y analizar el contrato.

Resultado esperado

Cada participante podra interactuar con su propio contrato inteligente desde el navegador, sin instalar herramientas adicionales.

Ruta de aprendizaje



Durante el taller

La teoría se usará como soporte directo para implementar el contrato del segundo día.

Dia 1

Fundamentos de blockchain, Ethereum y contratos inteligentes

Punto de partida: confianza digital

Blockchain no comienza con la tecnología.

Comienza con un problema de confianza, coordinacion y verificacion.

Confianza

¿A quien le creemos?

Registro

¿Donde queda evidencia?

Verificacion

¿Quien puede comprobarlo?

Ruta de la explicacion

Sistema tradicional → Registro compartido → Blockchain → Contratos inteligentes

¿Que problema busca resolver blockchain?

En muchos sistemas tradicionales, la informacion depende de una entidad central encargada de almacenar, validar y modificar los registros.

Esto concentra confianza, control y riesgo en un solo punto.

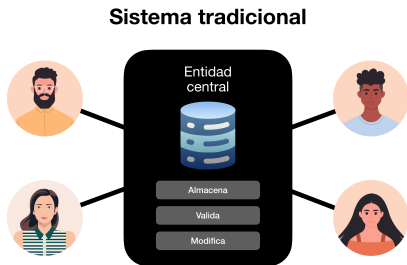
Problemas comunes

- ▶ Alteracion no autorizada de registros.
- ▶ Dependencia de una sola entidad.
- ▶ Dificultad para auditar cambios.
- ▶ Registros que no son facilmente verificables por terceros.

Pregunta guia

¿Como construir un registro compartido que distintos participantes puedan verificar sin depender completamente de un intermediario unico?

Sistema tradicional: confianza concentrada



Modelo centralizado

- ▶ Una entidad almacena.
- ▶ Una entidad valida.
- ▶ Una entidad modifica.
- ▶ Los usuarios dependen de ese intermediario.

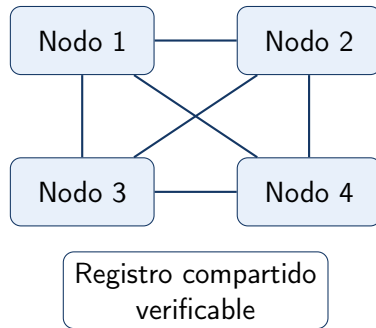
Riesgo

Si el registro cambia, falla o no es transparente, los participantes tienen pocas formas independientes de verificarlo.

Del registro central al registro compartido

Modelo blockchain

- ▶ Varios nodos mantienen una copia del registro.
- ▶ Las operaciones se validan bajo reglas comunes.
- ▶ El historial queda enlazado criptográficamente.
- ▶ La verificación no depende de un solo actor.

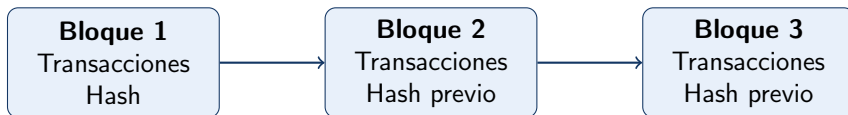


Conexión con el taller

Registraremos hashes de documentos para que su existencia y estado puedan verificarse posteriormente.

¿Que es una blockchain?

Una blockchain es un **registro compartido** donde las operaciones se agrupan en bloques enlazados criptograficamente.



Integridad

Alterar un dato cambia el hash asociado.

Trazabilidad

Las operaciones pueden revisarse despues.

Estado compartido

Varios nodos verifican una vision comun.

Hash: huella digital de la informacion

Idea

Una funcion hash transforma una entrada de cualquier tamano en una salida de longitud fija.

- ▶ Es determinista.
- ▶ Un pequeno cambio altera completamente la salida.
- ▶ Sirve para verificar integridad.

Ejemplo conceptual

Hola → E633f4...

hola → B221d9...

En nuestro contrato

No guardaremos el documento completo; guardaremos un identificador bytes32 derivado de su contenido.

Llaves, firmas y direcciones

Llave privada

Permite autorizar operaciones. Debe mantenerse secreta.

Firma digital

Demuestra que una cuenta autorizo una transaccion sin revelar la llave privada.

Llave publica / direccion

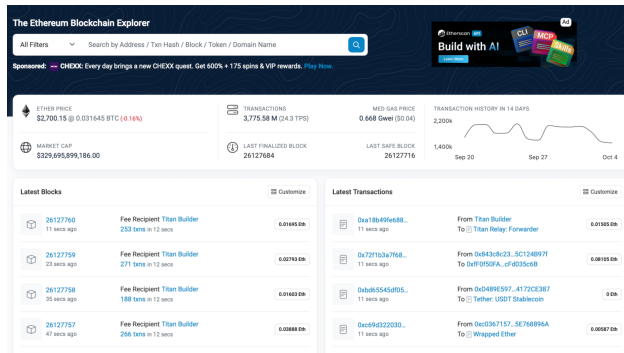
Permite identificar una cuenta y verificar operaciones firmadas.

Relacion con `msg.sender`

En Solidity, `msg.sender` identifica la cuenta o contrato que ejecuta la llamada actual.

Blockchain en la vida real: exploradores de bloques

Un explorador de bloques permite observar información pública de una red blockchain.



Permite revisar

- ▶ bloques recientes;
- ▶ transacciones;
- ▶ direcciones;
- ▶ comisiones y gas;
- ▶ contratos desplegados.

Conexion con Remix

Remix mostrara una version local y didactica de esta informacion en el historial de transacciones.

Anatomia basica de una transaccion

TRANSACTION ACTION

Titan Builder Call Data3267 Method on Titan Relay: Forwarder

Transaction Hash: 0xa18b49fe688f83d71be41921cd3684e22879043e2491eb05665427aa2640400

Status: Success

Block: 26127760 5 Block Confirmations

Timestamp: 58 secs ago Oct-05-2026 05:48:23 PM +UTC

Sponsored:



From: titanbuilder.eth (Titan Builder)

To: 0xFEEEEEE44046c3f61aBCC081ED918eF0de0a7FC (Titan Relay: Forwarder)

Internal Transactions:

All Transfers Net Transfers

Transfer 0.015053874819000264 ETH (\$40.65) From Titan Relay: Forwarder To 0x834bEEdc...4a0818Cc

Value: 0.015053874819000264 ETH (\$40.65)

Transaction Fee: 0.000018893289031008 ETH (\$0.05)

Gas Price: 0.650998864 Gwei (0.000000000650998864 ETH)

Campos importantes

- ▶ **Hash:** identificador de la transaccion.
- ▶ **Status:** exito o error.
- ▶ **From:** cuenta que firma.
- ▶ **To:** destino o contrato.
- ▶ **Fee:** costo pagado.
- ▶ **Gas:** trabajo computacional.

En la practica

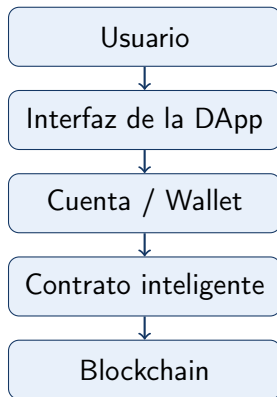
Veremos estos conceptos al desplegar, registrar y revocar en Remix VM.

¿Que es una DApp?

Una **DApp** es una aplicacion que combina una interfaz tradicional con logica y datos gestionados mediante blockchain.

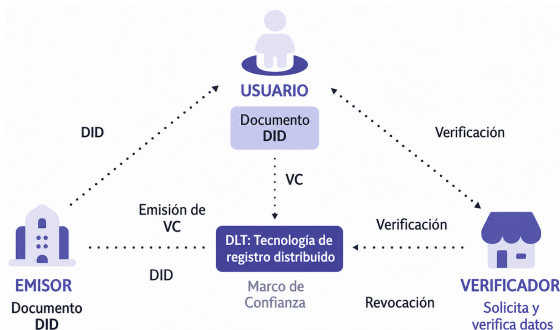
Idea clave

El usuario no suele interactuar directamente con la blockchain; lo hace mediante una interfaz, una cuenta o wallet y uno o mas contratos inteligentes.



Caso de uso: identidad autosoberana

La **identidad autosoberana (SSI)** busca que las personas tengan mayor control sobre sus identificadores y credenciales digitales, pudiendo presentarlos ante terceros para su verificación.



Idea clave

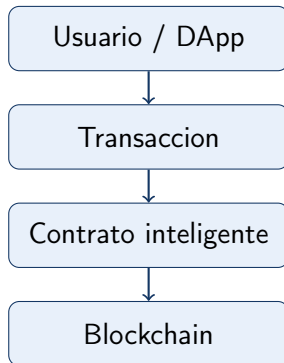
La blockchain o DLT puede actuar como **marco de confianza**, registrando información mínima y verificable, sin almacenar necesariamente las credenciales o documentos completos.

¿Donde entra un contrato inteligente?

Un **contrato inteligente** es un programa almacenado en blockchain que define reglas y operaciones ejecutables mediante transacciones.

Puede

- ▶ Recibir datos.
- ▶ Validar condiciones.
- ▶ Modificar su estado.
- ▶ Emitir eventos.
- ▶ Restringir acciones.



En el taller

El contrato registrara, consultara y revocara documentos.

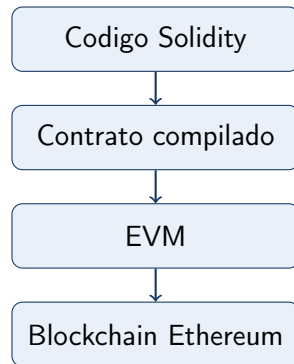
Ethereum y la EVM

Ethereum es una plataforma blockchain diseñada para ejecutar contratos inteligentes y aplicaciones descentralizadas.

Ethereum Virtual Machine

La EVM es el entorno de ejecución donde se procesa el código de los contratos inteligentes.

- ▶ Ejecuta instrucciones del contrato.
- ▶ Mantiene el estado.
- ▶ Procesa transacciones.
- ▶ Consume gas.



Cuentas, contratos y transacciones

EOA

Cuenta controlada por una llave privada. Inicia transacciones.

Contrato

Cuenta con código asociado. Ejecuta reglas al recibir llamadas.

Transaccion

Operación firmada que puede modificar el estado de la blockchain.

Idea para la práctica

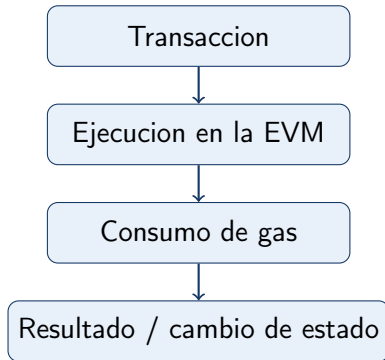
Al registrar y revocar documentos enviaremos transacciones. Al consultar información usaremos llamadas de lectura.

¿Que es el gas?

El **gas** representa el costo computacional de ejecutar operaciones en Ethereum.

Diferencia importante

- ▶ Una llamada externa a una funcion `view` no genera una transaccion y normalmente no requiere pagar gas.
- ▶ Modificar el estado si requiere una transaccion y consume gas.



Fundamentos de Solidity que usaremos

`struct`

Agrupar datos relacionados.

`address`

Representa cuentas o contratos.

`view`

Lee estado sin modificarlo.

`mapping`

Relaciona una clave con un valor.

`msg.sender`

Identifica quien ejecuta una funcion.

`pure`

Calcula sin leer estado.

`bytes32`

Formato comun para hashes.

`require`

Detiene la ejecucion si una condicion falla.

Eventos

Permiten observar acciones del contrato.

Seguridad basica en contratos

Validar entradas

No aceptar datos vacios, duplicados o inconsistentes.

Control de acceso

No cualquier cuenta debe ejecutar cualquier funcion.

Estado claro

Evitar estados imposibles o ambiguos.

Errores comunes

- ▶ Olvidar restricciones de propietario.
- ▶ Guardar informacion innecesaria on-chain.
- ▶ No emitir eventos relevantes.
- ▶ No diferenciar lectura y escritura.

Cierre del Dia 1: que debe quedar claro

Mapa mental

Documento → Hash → Contrato → Registro → Consulta → Revocacion

Preparacion tecnica

- ▶ Tener acceso a Internet.
- ▶ Abrir el repositorio en GitHub.
- ▶ Verificar que Remix IDE cargue.

Para el Dia 2

No sera necesario instalar Solidity, Node.js, MetaMask ni una blockchain local.
Trabajaremos en Remix VM.

Dia 2

Practica guiada con Solidity y Remix IDE

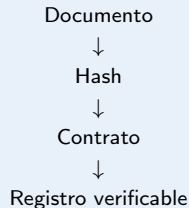
¿Que vamos a construir?

Implementaremos un contrato inteligente para registrar informacion verificable sobre documentos.

Funciones principales

- ▶ Registrar un documento mediante su hash.
- ▶ Consultar quien realizo el registro.
- ▶ Verificar si el documento sigue vigente.
- ▶ Revocar un registro.
- ▶ Emitir eventos observables desde Remix.

Flujo



Idea clave

No almacenaremos el archivo completo en blockchain.

Metodo de trabajo del Dia 2

No sera solo copiar codigo

La practica se guiara por ciclos pequenos:

1. Leer un fragmento del contrato.
2. Entender que regla implementa.
3. Ejecutarlo o validarlo en Remix.
4. Observar estado, eventos y gas.

Razon didactica

Primero explicamos lo minimo necesario y luego lo probamos de inmediato.

Resultado esperado

Cada participante no solo ejecuta el contrato: tambien entiende que hace cada bloque.

Acceso al material del taller

Repositorio

`github.com/NullAstra404/
rci5-contratos-inteligentes`

Abrir en Remix

El boton **Abrir en Remix IDE** carga directamente el contrato completo en el navegador.

Importante

Cada participante trabajara en su propia instancia de Remix VM.



Escanea para abrir el repositorio en GitHub

Mapa del contrato que revisaremos

Datos y almacenamiento

- ▶ `struct Documento`
- ▶ `mapping(bytes32 => Documento)`
- ▶ Campos: hash, propietario, fecha, estado

Observabilidad

- ▶ `DocumentoRegistrado`
- ▶ `DocumentoRevocado`

Funciones

- ▶ `calcularHash()`
- ▶ `registrarDocumento()`
- ▶ `consultarDocumento()`
- ▶ `revocarDocumento()`

Reglas

Validar entradas, evitar duplicados, consultar estado y restringir revocacion al propietario.

Bloque 1: contrato inicial y documentacion

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 /**
5  * @title RegistroDocumentos
6  * @notice Contrato didactico para registrar,
7  * consultar y revocar identificadores de
8  * documentos.
9  */
10 contract RegistroDocumentos {
11 }
```

Que explicamos

- ▶ SPDX: licencia del codigo fuente.
- ▶ pragma: version del compilador.
- ▶ NatSpec: documentacion tecnica.
- ▶ contract: unidad desplegable.

Idea tecnica

Al desplegar, Remix crea una instancia del contrato con direccion propia dentro de Remix VM.

Bloque 2: representar un documento con struct

```
1 struct Documento {  
2     bytes32 hashDocumento;  
3     address propietario;  
4     uint256 fechaRegistro;  
5     bool vigente;  
6 }
```

Que modela

- ▶ bytes32: identificador de 32 bytes; aqui representa el hash.
- ▶ address: cuenta que registra el documento.
- ▶ uint256: timestamp del registro.
- ▶ bool: estado logico del registro.

Punto importante

El struct define la forma del dato; por si solo no guarda registros.

Bloque 3: almacenamiento con mapping

```
1 mapping(bytes32 => Documento)  
2     private documentos;
```

Relacion

hash → Documento

Por que mapping

El acceso principal sera buscar un documento conociendo su hash.

Cuidado con private

private restringe el acceso desde Solidity, pero no hace secreta la informacion on-chain.

Bloque 4: eventos para observar acciones

```
1 event DocumentoRegistrado(  
2     bytes32 indexed hashDocumento,  
3     address indexed propietario,  
4     uint256 fechaRegistro  
5 );  
6  
7 event DocumentoRevocado(  
8     bytes32 indexed hashDocumento,  
9     address indexed propietario  
10 );
```

Que hacen

Definen logs que se emitiran cuando ocurra una accion importante.

indexed

Permite filtrar eventos por hash o propietario.

Estado vs. evento

El mapping guarda estado; los eventos dejan trazabilidad observable.

Bloque 5: funcion auxiliar calcularHash

```
1 function calcularHash(  
2     string memory _texto  
3 )  
4     public  
5     pure  
6     returns (bytes32)  
7 {  
8     return keccak256(  
9         abi.encodePacked(_texto)  
10    );  
11 }
```

Que hace

Convierte un texto en bytes y calcula un hash Keccak-256.

pure

No lee ni modifica el estado del contrato; solo calcula.

Nota arquitectonica

En un sistema real, el hash del archivo normalmente se calcula fuera de blockchain.

Bloque 6: registrar - validaciones

```
1 function registrarDocumento(bytes32 _hash)
2     public
3 {
4     require(
5         _hash != bytes32(0),
6         "Hash no valido"
7     );
8     require(
9         documentos[_hash].propietario ==
10         address(0),
11         "Documento ya registrado"
12     );
13     // ... guardar documento
```

Primera regla

No aceptar un hash vacio.

Segunda regla

No permitir registrar dos veces el mismo hash.

address(0)

Se usa como valor por defecto para detectar que aun no hay propietario.

Bloque 6: registrar - escritura y evento

```
1 documentos[_hash] = Documento({
2     hashDocumento: _hash,
3     propietario: msg.sender,
4     fechaRegistro: block.timestamp,
5     vigente: true
6 });
7
8 emit DocumentoRegistrado(
9     _hash,
10    msg.sender,
11    block.timestamp
12 );
```

Que se guarda

- ▶ El hash.
- ▶ El propietario: msg.sender.
- ▶ La fecha: block.timestamp.
- ▶ Estado inicial: true.

Costo

Modificar storage requiere transaccion y consume gas.

Bloque 7: consultar con view

```
1 function consultarDocumento(bytes32 _hash)
2     public
3     view
4     returns (
5         address propietario,
6         uint256 fechaRegistro,
7         bool vigente
8     )
9 {
10     require(
11         documentos[_hash].propietario != address
12         (0),
13         "Documento no registrado"
14     );
15     Documento memory doc = documentos[_hash];
16     return (doc.propietario, doc.fechaRegistro,
17         doc.vigente);
18 }
```

view

Lee el estado, pero no lo modifica.

memory

Copia temporal para devolver los datos de forma ordenada.

En Remix

Aparecera como llamada de lectura.

Bloque 8: revocar con control de acceso

```
1 function revocarDocumento(bytes32 _hash)
2     public
3 {
4     require(documentos[_hash].propietario !=
5         address(0),
6         "Documento no registrado");
7     require(documentos[_hash].propietario ==
8         msg.sender,
9         "No autorizado");
10    require(documentos[_hash].vigente,
11        "Documento ya revocado");
12
13    documentos[_hash].vigente = false;
14    emit DocumentoRevocado(_hash, msg.sender);
15 }
```

Validaciones

- ▶ Existe el documento.
- ▶ Lo ejecuta el propietario.
- ▶ Sigue vigente.

Resultado

Revocar no borra: cambia vigente a false.

Control de acceso con `msg.sender`

Escenario permitido

Cuenta A registra



Cuenta A revoca

Permitido

Escenario no autorizado

Cuenta A registra



Cuenta B intenta revocar

No autorizado

Regla implementada

```
documentos[_hash].propietario == msg.sender
```

Contrato completo: estructura final

Datos

- ▶ `struct Documento`
- ▶ `mapping(bytes32 => Documento)`

Funciones

- ▶ `calcularHash()`
- ▶ `registrarDocumento()`
- ▶ `consultarDocumento()`
- ▶ `revocarDocumento()`

Validaciones

- ▶ Hash valido.
- ▶ Documento existente o no existente.
- ▶ Estado vigente.
- ▶ Control de acceso.

Eventos

- ▶ `DocumentoRegistrado`
- ▶ `DocumentoRevocado`

Ahora si: prueba guiada en Remix

Objetivo de esta segunda parte

Usar el contrato completo para observar lo que acabamos de explicar: compilacion, despliegue, llamadas, transacciones, control de acceso, eventos y gas.

Rol del instructor

Guiar paso a paso y detenerse en errores utiles.

Rol del participante

Ejecutar cada paso en su propio workspace de Remix.

Practica en Remix: flujo operativo

1. Abrir el contrato desde GitHub.
2. Compilar `RegistroDocumentos.sol`.
3. Seleccionar **Remix VM**.
4. Desplegar el contrato.
5. Generar un hash.
6. Registrar el documento.
7. Consultar el estado.
8. Probar revocacion y eventos.

Regla de seguridad

No se usara wallet real, no se usara dinero real y no se conectara a una red publica.

Compilar y desplegar

Compilar

- ▶ Abrir **Solidity Compiler**.
- ▶ Verificar version 0.8.x compatible.
- ▶ Pulsar **Compile**.
- ▶ Confirmar que no hay errores.

Desplegar

- ▶ Abrir **Deploy & Run Transactions**.
- ▶ Seleccionar **Remix VM**.
- ▶ Mantener **Value = 0**.
- ▶ Pulsar **Deploy**.

Prueba 1: hash, registro y consulta

Generar hash

Ejecutar:

```
calcularHash("Documento-RCI-001")
```

Copiar el valor bytes32 devuelto.

Registrar

Con **Cuenta A**, ejecutar:

```
registrarDocumento(hash)
```

Consultar

Ejecutar:

```
consultarDocumento(hash)
```

Resultado esperado:

- ▶ propietario = Cuenta A
- ▶ fechaRegistro = timestamp
- ▶ vigente = true

Prueba 2: revocacion y control de acceso

Intento no autorizado

Cambiar a **Cuenta B** y ejecutar:

```
revocarDocumento(hash)
```

Resultado esperado:

No autorizado

Revocacion correcta

Volver a **Cuenta A** y ejecutar:

```
revocarDocumento(hash)
```

Consultar de nuevo:

```
vigente = false
```

Eventos, transacciones y gas

Eventos

- ▶ DocumentoRegistrado
- ▶ DocumentoRevocado

Transacciones

- ▶ Deploy.
- ▶ Registrar.
- ▶ Revocar.

Gas

- ▶ Desplegar consume gas.
- ▶ Modificar estado consume gas.
- ▶ Consultar con view no cambia estado.

Aprendizaje

Remix permite observar el comportamiento de una DApp sin conectarse a una red publica.

Errores comunes durante la practica

Compilacion

- ▶ Llaves { } faltantes.
- ▶ Punto y coma faltante.
- ▶ Version de compilador incompatible.

Ejecucion

- ▶ Enviar un hash incompleto.
- ▶ Registrar dos veces el mismo hash.
- ▶ Revocar desde otra cuenta.

Como resolver

- ▶ Leer el mensaje de error.
- ▶ Verificar la cuenta activa.
- ▶ Revisar el historial de transacciones.
- ▶ Comparar con `RegistroDocumentos.sol` del repositorio.

De contrato a DApp real

Lo que hicimos

- ▶ Definimos reglas en Solidity.
- ▶ Desplegamos en una EVM local.
- ▶ Ejecutamos transacciones.
- ▶ Observamos eventos y gas.

Lo que faltaria en produccion

- ▶ Frontend.
- ▶ Wallet real.
- ▶ Red de prueba o mainnet.
- ▶ Auditoria de seguridad.
- ▶ Gestion de datos off-chain.

Conclusiones

Ideas principales

- ▶ Blockchain permite registros compartidos, verificables y trazables.
- ▶ Ethereum ejecuta contratos inteligentes mediante la EVM.
- ▶ Solidity permite expresar reglas de negocio como código.
- ▶ Remix IDE facilita compilar, desplegar y probar sin instalaciones locales.
- ▶ Un contrato debe validar entradas, controlar permisos y emitir eventos relevantes.

Documento → Hash → Contrato → Registro verificable

Repositorio del taller

[github.com/NullAstra404/
rci5-contratos-inteligentes](https://github.com/NullAstra404/rci5-contratos-inteligentes)

Contenido

- ▶ Contrato completo.
- ▶ Contrato base.
- ▶ Guia practica.
- ▶ Acceso directo a Remix IDE.



Repositorio del taller

Datos de contacto

Israel Jaudy Perez Bermudez

`israelj.perezb@inaoe.mx`

Armando Rivera Castillo

`armando.riverac@inaoe.mx`

Gracias